Чисельні методи

Лабораторна робота №2

Методи чисельного розв’язку систем лінійних рівнянь.

Виконав:

Студент КН-24

Бевзюк Н.С.

2026

Варіант **1**

1. Мета роботи.

Набути практичних навичок розроблення алгоритмів і програм чисельних методів розв'язання систем лінійних рівнянь методом Зейделя, навчитися приводити систему до ітераційного вигляду з умовами збіжності, реалізувати метод та візуалізувати траєкторію пошуку.

2. Завдання.

1) Взяти задану систему .

2) Перевірити, що на діагоналі немає нульових елементів; якщо є — спробувати переставити рядки так, щоб на діагоналі були ненульові значення.

3) Домогтися строгого діагонального домінування по рядках через елементарні перетворення, щоб забезпечити збіжність методу Зейделя.

4) Виконати програмний код з точністю $\varepsilon = 0{,}001$.

5) Побудувати траєкторію пошуку

3. Розрахункові формули чисельного методу.

Метод Зейделя є одним із фундаментальних ітераційних алгоритмів для розв'язання систем лінійних алгебраїчних рівнянь. Він виник як логічне вдосконалення методу простої ітерації (методу Якобі). Головна особливість методу полягає у принципі свіжих даних: як тільки ми обчислюємо нове значення однієї змінної, воно негайно використовується для розрахунку наступних змінних у межах тієї самої ітерації. У класичному методі Якобі всі значення оновлюються одночасно лише наприкінці циклу, що робить метод Зейделя динамічнішим та зазвичай швидшим у досягненні результату.

Процес розв'язання починається з вибору початкового наближення — довільного вектора значень, який вважається приблизним розв'язком. Оскільки метод є ітераційним, він не гарантує отримання результату для будь-якої матриці. Для того, щоб значення змінних з кожним кроком наближалися до істинного розв'язку, а не безконтрольно зростали, система має задовольняти діагональне переважання. Це означає, що в кожному рядку матриці коефіцієнт, що стоїть на головній діагоналі, має бути більшим за суму модулів усіх інших коефіцієнтів цього рядка.

Головною перевагою методу Зейделя є швидкість збіжності, яка зазвичай удвічі вища, ніж у методу Якобі. Крім того, він потребує менше оперативної пам'яті, оскільки не вимагає одночасного зберігання старого та нового векторів розв'язку

— оновлення відбувається безпосередньо у тій самій ділянці пам'яті. Метод завершується, коли різниця між значеннями змінних на двох сусідніх ітераціях стає меншою за задану точність.

4. Перетворена система із урахуванням умов збіжності.

Початкова система рівнянь $\begin{cases} 2.1x_1 + 1.6x_2 + 3.3x_3 = 2.1 \\ 3.5x_1 - 1.7x_2 + 2.8x_3 = 1.7 \\ 4.1x_1 + 5.8x_2 - 1.7x_3 = 0.8 \end{cases}$, перевіряється на діагональне переваження:

|2.1| < |1.6| + |3.3|

|-1.7| < |3.5| + |2.8|

|-1.7| < |4.1| + |-1.7|

Не виконується ні для одного рівняння. Отже застосовуємо метод елементарних перетворень:

1 рівняння утворимо як сума 2 та 3 рівняннь, оскільки при них перший коефіцієнт є найбільшим: $(3,5 + 4,1)x_1 + (-1,7 + 5,8)x_2 + (2,8 - 1,7)x_3 = 1,7 + 0,8$. Отримуємо $7,6x_1 + 4,1x_2 + 1,1x_3 = 2,5$, і шляхом перевірки |7,6| > |4,1| + |1,1| бачимо що умова виконується.

2 рівняння сформується за рахунок віднімання 3 рівняння від 2-го, щоб досягти найбільшого другого коефіцієнту: $(4,1 - 3,5)x_1 + (5,8 - (-1,7))x_2 + (-1,7 - 2,8)x_3 = 0,8 - 1,7$ отримуємо $0,6x_1 + 7,5x_2 - 4,5x_3 = -0,9$ після перевірки |7,5| > |0,6| + |-4,5| наочно бачимо виконання умови.

3 рівняння для створення сильного третього коефіцієнту утворюємо за рахунок множення 1 рівняння на три і віднімання цього рівняння від 3-го:

$(3 * 2,1 - 4,1)x_1 + (3 * 1,6 - 5,8)x_2 + (3 * 3,3 - (-1,7))x_3 = 3 * 2,1 - 0,8$

$(6,3 - 4,1)x_1 + (4,8 - 5,8)x_2 + (9,9 + 1,7)x_3 = 6,3 - 0,8$ отримуємо $2,2x_1 - 1,0x_2 + 11,6x_3 = 5,5$ перевіряємо це рівняння |11,6| > |2,2| + |-1,0|, умова виконується.

Отже нова система рівнянь, яка є гарантовано збіжною

$$\begin{cases} 7.6x_1 + 4.1x_2 + 1.1x_3 = 2.5 \\ 0.6x_1 + 7.5x_2 - 4.5x_3 = -0.9 \\ 2.2x_1 - 1.0x_2 + 11.6x_3 = 5.5 \end{cases}$$

5. Текст програми.

ПОЧАТОК КОДУ

```
import tkinter as tk
from tkinter import messagebox, ttk
import numpy as np

# Перевірка діагонального переваження
def is_diagonally_dominant(A):
    for i in range(3):
        diag = abs(A[i][i])
        others = sum(abs(A[i][j]) for j in range(3) if j != i)
        if diag <= others:
            return False
    return True

# Метод Зейделя
def seidel(A, b, eps=0.001, max_iter=100):
    x = np.zeros(3)
    history = [x.copy()]
    for _ in range(max_iter):
        x_new = x.copy()
        for i in range(3):
            s1 = sum(A[i][j]*x_new[j] for j in range(i))
            s2 = sum(A[i][j]*x[j] for j in range(i+1,3))
            x_new[i] = (b[i] - s1 - s2) / A[i][i]
        history.append(x_new.copy())
        if np.linalg.norm(x_new-x,ord=np.inf) < eps:
            return x_new, history
        x = x_new
    return x, history

# Малювання траєкторії
def draw_graphs(history):
    for widget in graph_frame.winfo_children():
        widget.destroy()

    pairs = [("x1","x2"),("x1","x3"),("x2","x3")]
    colors = ["blue","green","purple"]

    for idx, (i,j) in enumerate([(0,1),(0,2),(1,2)]):
        c =
tk.Canvas(graph_frame,width=300,height=250,bg="white",highlightthickness=1,high
lightbackground="black")
        c.grid(row=0,column=idx,padx=10)
```

```
        xs = [h[i] for h in history]
        ys = [h[j] for h in history]

        min_x,max_x = min(xs),max(xs)
        min_y,max_y = min(ys),max(ys)

        if max_x-min_x==0: max_x+=1
        if max_y-min_y==0: max_y+=1

        def scale_x(x): return 30 + (x-min_x)/(max_x-min_x)*(240)
        def scale_y(y): return 220 - (y-min_y)/(max_y-min_y)*(180)

        c.create_line(30,220,270,220,width=2)
        c.create_line(30,40,30,220,width=2)
        c.create_text(150,230,text=pairs[idx][0])
        c.create_text(10,130,text=pairs[idx][1],angle=90)

        for k in range(len(xs)-1):
            x1,y1 = scale_x(xs[k]),scale_y(ys[k])
            x2,y2 = scale_x(xs[k+1]),scale_y(ys[k+1])
            c.create_line(x1,y1,x2,y2,fill=colors[idx],width=2)
            c.create_oval(x1-3,y1-3,x1+3,y1+3,fill="red")
            c.create_text(x1-5,y1-10,text=str(k),font=("Arial",7))

        x_last,y_last = scale_x(xs[-1]),scale_y(ys[-1])
        c.create_oval(x_last-4,y_last-4,x_last+4,y_last+4,fill="green")
        c.create_text(x_last-5,y_last-10,text=str(len(xs)-1),font=("Arial",7))

# Заповнення таблиці ітерацій
def fill_table(history):
    for row in table.get_children():
        table.delete(row)
    for i,h in enumerate(history):
        table.insert(
            "",
            "end",
            values=(
                i,
                f"{h[0]:.5f}",
                f"{h[1]:.5f}",
                f"{h[2]:.5f}"
            )
        )
```

```
# Розв'язання
def solve():
    try:
        def parse(e):
            return float(e.get().replace(",", "."))

        A = np.array([
            [parse(a11),parse(a12),parse(a13)],
            [parse(a21),parse(a22),parse(a23)],
            [parse(a31),parse(a32),parse(a33)]
        ])

        b = np.array([
            parse(b1),
            parse(b2),
            parse(b3)
        ])

        eps = parse(eps_entry)

        if not is_diagonally_dominant(A):
            messagebox.showerror(
                "Помилка",
                "Матриця не має діагонального переваження.\n"
                "Метод Зейделя може не збігатися."
            )
            return
        solution,history = seidel(A,b,eps)

        result_var.set(
            f"x1 = {solution[0]:.5f}\n"
            f"x2 = {solution[1]:.5f}\n"
            f"x3 = {solution[2]:.5f}"
        )
        draw_graphs(history)
        fill_table(history)

    except Exception as e:
        messagebox.showerror("Помилка",str(e))


# GUI
```

```
root = tk.Tk()
root.title("Метод Зейделя")

top = tk.Frame(root)
top.pack(pady=10)

tk.Label(top,text="Вводьте числа через крапку
(.)").grid(row=0,column=0,columnspan=4)

tk.Label(top,text="x1").grid(row=1,column=0)
tk.Label(top,text="x2").grid(row=1,column=1)
tk.Label(top,text="x3").grid(row=1,column=2)
tk.Label(top,text="b").grid(row=1,column=3)

def entry(r,c):
    e=tk.Entry(top,width=7)
    e.grid(row=r,column=c,padx=3,pady=3)
    return e

a11=entry(2,0)
a12=entry(2,1)
a13=entry(2,2)
b1=entry(2,3)
a21=entry(3,0)
a22=entry(3,1)
a23=entry(3,2)
b2=entry(3,3)
a31=entry(4,0)
a32=entry(4,1)
a33=entry(4,2)
b3=entry(4,3)

tk.Label(root,text="Точність ε").pack()

eps_entry=tk.Entry(root,width=10)
eps_entry.insert(0,"0.001")
eps_entry.pack()

tk.Button(
    root,
    text="Розв'язати",
    command=solve,
    height=2
```

```
).pack(pady=10)

result_var=tk.StringVar()

tk.Label(
    root,
    textvariable=result_var,
    font=("Arial",12)
).pack()

# графік
graph_frame = tk.Frame(root)
graph_frame.pack(pady=10)

# Таблиця ітерацій
columns=("k","x1","x2","x3")
table=ttk.Treeview(root,columns=columns,show="headings",height=8)

for c in columns:
    table.heading(c,text=c)
    table.column(c,width=100,anchor="center")

table.pack(pady=10)

root.mainloop()
```

КІНЕЦЬ КОДУ

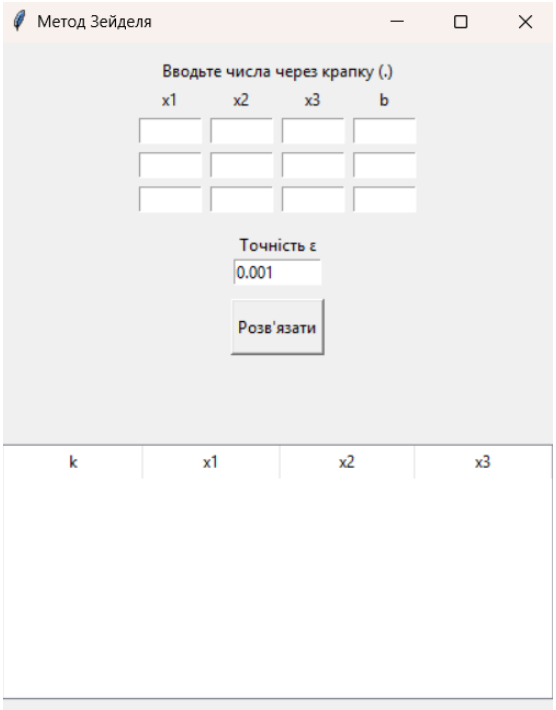
6. Результат роботи програми та траєкторія пошуку.

Дані:

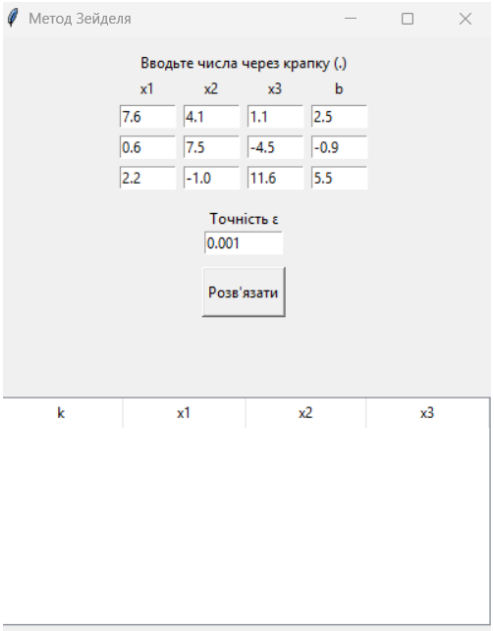
Система рівняннь - $\begin{cases} 7.6x_1 + 4.1x_2 + 1.1x_3 = 2.5 \\ 0.6x_1 + 7.5x_2 - 4.5x_3 = -0.9 \\ 2.2x_1 - 1.0x_2 + 11.6x_3 = 5.5 \end{cases}$

Точність ε - 0,001

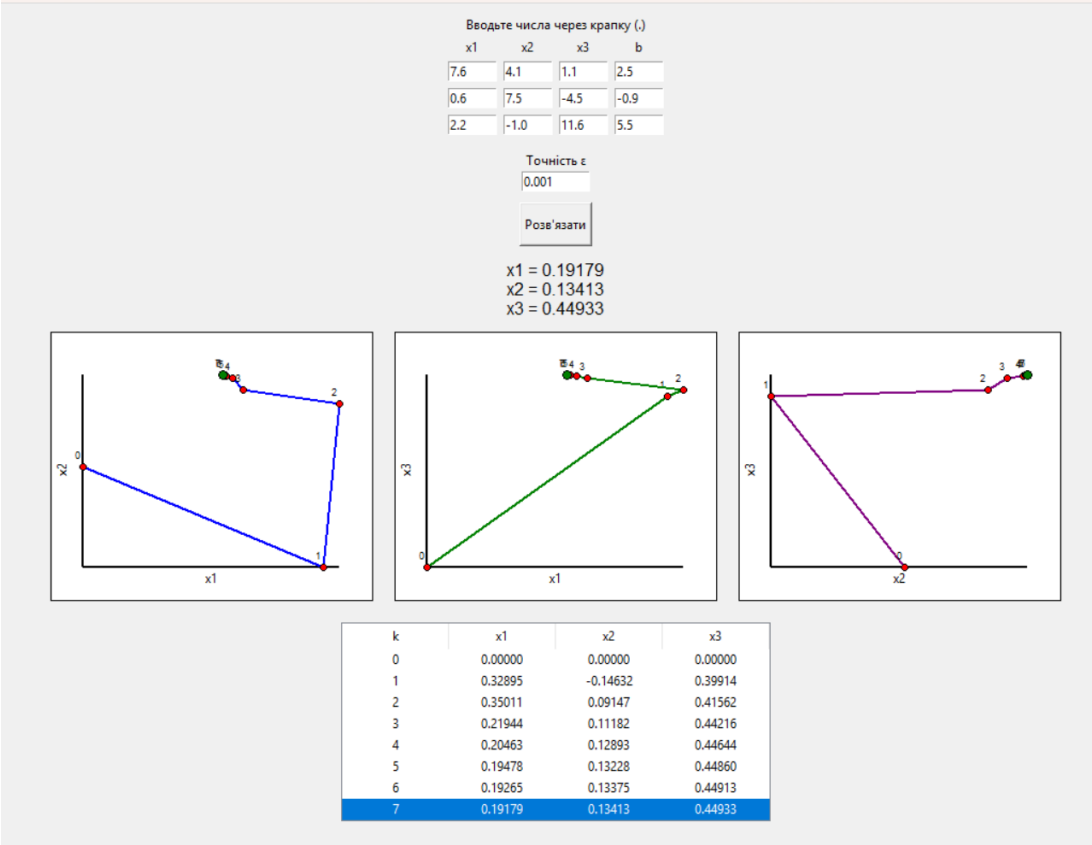
Запуск програми:



Внесення даних з системи рівняннь та перевірка точності:



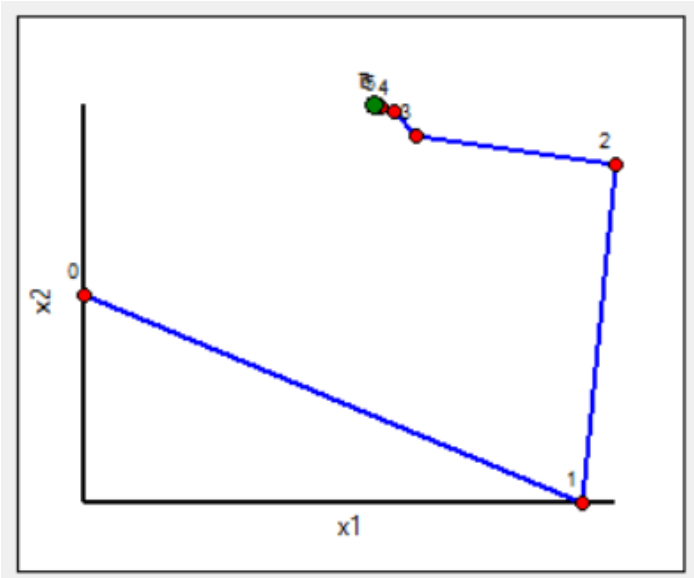
Натискання на кнопку “ Розв’язати”:

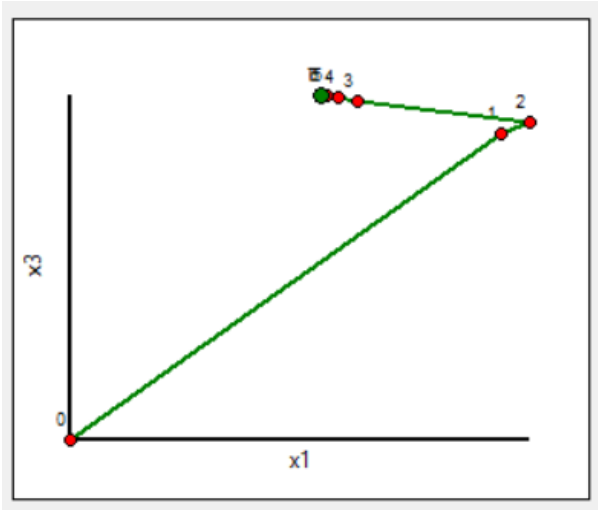


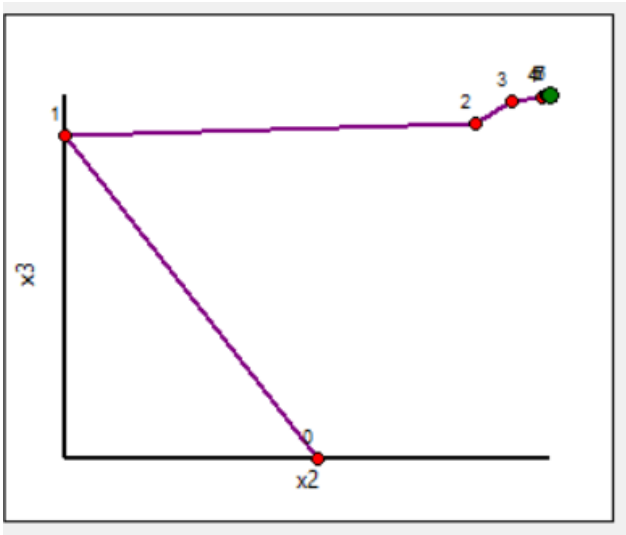
Перегляд результату – відповідь:

x1 = 0.19179
x2 = 0.13413
x3 = 0.44933

Перегляд результату – траєкторії пошуку:







Перегляд результату – таблиця ітерацій:

k	x1	x2	x3
0	0.00000	0.00000	0.00000
1	0.32895	-0.14632	0.39914
2	0.35011	0.09147	0.41562
3	0.21944	0.11182	0.44216
4	0.20463	0.12893	0.44644
5	0.19478	0.13228	0.44860
6	0.19265	0.13375	0.44913
7	0.19179	0.13413	0.44933

Перевірка результатів:

Знайдені значення: x_1 = **0.19179**, x_2 = **0.13413**, x_3 = **0.44933** підставимо в діагонально переважену систему $\begin{cases} 7.6x_1 + 4.1x_2 + 1.1x_3 = 2.5 \\ 0.6x_1 + 7.5x_2 - 4.5x_3 = -0.9 \\ 2.2x_1 - 1.0x_2 + 11.6x_3 = 5.5 \end{cases}$ та підставимо

$$\begin{cases} 7.6 \cdot 0.19179 + 4.1 \cdot 0.13413 + 1.1 \cdot 0.44933 \\ 0.6 \cdot 0.19179 + 7.5 \cdot 0.13413 - 4.5 \cdot 0.44933 \\ 2.2 \cdot 0.19179 - 1.0 \cdot 0.13413 + 11.6 \cdot 0.44933 \end{cases} = \begin{cases} 2.50180 \\ -0.900936 \\ 5.500036 \end{cases}$$

, що зважаючи на задану точність – відповідає умові системи

Тепер отриманні значення підставляємо в вихідну систему, що задана за умовою $\begin{cases} 2.1x_1 + 1.6x_2 + 3.3x_3 = 2.1 \\ 3.5x_1 - 1.7x_2 + 2.8x_3 = 1.7 \\ 4.1x_1 + 5.8x_2 - 1.7x_3 = 0.8 \end{cases} = \begin{cases} 0.402759 + 0.214608 + 1.482789 \\ 0.671265 - 0.228021 + 1.258124 \\ 0.786339 + 0.777954 - 0.763861 \end{cases}$

$= \begin{cases} 2.100156 \\ 1.701368 \\ 0.800432 \end{cases}$, з урахуванням точності – підрахунки співпадають з відповіддю.

7. Висновки.

У результаті виконання лабораторної роботи було здобуто практичні навички розроблення алгоритмів чисельного розв'язання систем лінійних рівнянь. Основним етапом роботи стала попередня підготовка системи: оскільки вихідна матриця не задовольняла умові діагонального переважання, було застосовано елементарні перетворення шляхом лінійних комбінацій рядків. Це дозволило сформувати нову систему, де модулі діагональних елементів перевищують суму модулів інших коефіцієнтів у відповідних рядках, що є ключовою гарантією збіжності ітераційного процесу.

Практична реалізація методу Зейделя підтвердила його переваги у швидкості досягнення результату. Використання щойно обчислених значень змінних безпосередньо всередині поточної ітерації дозволило значно прискорити процес уточнення коренів порівняно з методом простої ітерації. Завдяки цьому було досягнуто заданої точності 0,001 за оптимальну кількість кроків, що демонструє обчислювальну стійкість та ефективність алгоритму для розв'язання прикладних задач.

На завершення можна зробити висновок, що оволодіння ітераційними методами, зокрема методом Зейделя, є необхідним для вирішення систем великої розмірності, де прямі методи можуть бути надто ресурсомісткими. Отриманий досвід дозволяє не лише програмувати математичні моделі, а й критично оцінювати структуру матриць на предмет їхньої чисельної придатності до ітераційного розв'язку.